

Deep Dive into Python OOP: How Inheritance Affects the Design of Model Compression

1. Introduction

In recent years, the field of artificial intelligence has witnessed a surge in overparameterized models. Large Language Models (LLMs), some exceeding 100 billion parameters, have become the standard. Consequently, model compression has emerged as a crucial area of research to facilitate pre-training, fine-tuning, and inference on resource-constrained edge devices. Prominent compression methodologies include Pruning, which removes low-impact weights; Quantization, which reduces precision from 32-bit floating point to 8-bit or 4-bit; and Knowledge Distillation (KD), where a compact student model learns the distribution of a large teacher model.

However, implementing these diverse compression techniques within a single software system often leads to highly coupled and unmaintainable "spaghetti code." This report demonstrates that Inheritance, a core pillar of Python Object-Oriented Programming (OOP), provides a sophisticated solution to these challenges, enabling the construction of a scalable and modular compression framework.

2. Inheritance

Inheritance is a mechanism that allows a new class (child) to derive attributes and methods from an existing class (parent), thereby maximizing code reusability. By overriding inherited methods, child classes can implement specific behaviors tailored to their requirements. This naturally facilitates polymorphism, allowing the system to process different objects through a uniform interface. Furthermore, inheritance supports the Open-Closed Principle (OCP) in software engineering; the base system remains closed to modification (preserving stability) while being open to extension through new subclasses.

3. Implementation

The experiments were conducted by implementing a forward function representing a simplified neural network layer.

Experiment 1 implements basic, pruning, and quantization logic using a

procedural approach without inheritance.

Experiment 2 implements the same functionality utilizing inheritance and polymorphism.

A. Experiment 1: Procedural Approach

```
def forward(x, weights, compression, threshold=0.0):  
    if compression == "basic":  
        return [w * x for w in weights]  
    elif compression == "pruning":  
        return [(w if abs(w) > threshold else 0.0) * x for w in weights]  
    elif compression == "quantization":  
        return [round(w) * x for w in weights]  
    else:  
        raise ValueError("Unknown layer type")  
  
weights = [0.1, -0.8, 0.4, -0.2]  
  
print(f"basic forward : {forward(2.0, weights, \"basic\", threshold=0.3)}")  
print(f"pruning forward : {forward(2.0, weights, \"pruning\", threshold=0.3)}")  
print(f"quantization forward : {forward(2.0, weights, \"quantization\", threshold=0.3)}")  
  
basic forward : [0.2, -1.6, 0.8, -0.4]  
pruning forward : [0.0, -1.6, 0.8, 0.0]  
quantization forward : [0.0, -2.0, 0.0, 0.0]
```

(Figure 1. Procedural approach of model toy compression code(up) and result(down))

B. Experiment 2: Object-Oriented Approach (Inheritance)

```
class BaseLayer:  
    def __init__(self, name, weights):  
        self.name = name  
        self.weights = weights  
  
    def forward(self, x):  
        raise NotImplementedError("Subclasses must implement forward method.")  
  
class BasicLayer(BaseLayer):  
    def forward(self, x):  
        return [w * x for w in self.weights]  
  
class PrunedLayer(BaseLayer):  
    def __init__(self, name, weights, threshold=0.5):  
        super().__init__(name, weights)  
        self.threshold = threshold  
  
    def forward(self, x):  
        return [(w if abs(w) > self.threshold else 0.0) * x for w in self.weights]  
  
class QuantizedLayer(BaseLayer):  
    def forward(self, x):  
        return [round(w) * x for w in self.weights]  
  
layers = [  
    BasicLayer("basic forward", [0.1, -0.8, 0.4, -0.2]),  
    PrunedLayer("pruning forward", [0.1, -0.8, 0.4, -0.2], threshold=0.3),  
    QuantizedLayer("quantization forward", [0.1, -0.8, 0.4, -0.2])  
]  
  
for layer in layers:  
    print(f"{layer.name} : {layer.forward(2.0)}")
```

```
basic forward : [0.2, -1.6, 0.8, -0.4]  
pruning forward : [0.0, -1.6, 0.8, 0.0]  
quantization forward : [0.0, -2.0, 0.0, 0.0]
```

(Figure 2. Object-Oriented approach using inheritance of model toy compression code(up) and result(down))

C. Comparative Analysis

In Experiment 1, each compression case necessitates explicit conditional statements (if/elif/else). Conversely, Experiment 2 eliminates these branches by overriding the parent class methods. This allows the execution pipeline to invoke `layer.forward(2.0)` regardless of the specific layer type.

If a new technique, such as KD-based compression, were to be added, the procedural approach would require modifying the existing forward function, risking regression errors. In an OOP paradigm, one simply implements a new `KDLayer` class without altering the existing codebase. Finally, the OOP approach enables state encapsulation; while Experiment 1 requires passing the threshold as an argument every time, the `PrunedLayer` object maintains its own threshold as an internal attribute, leading to a cleaner and more robust interface.

4. Discussion

While the code for Experiment 2 is longer for a simple operation, the benefits in scalability and safety are far greater when dealing with complex architectures like LLMs. The flexibility of an inheritance-based design is essential for modern AI research, as seen in the foundational work of Han et al. (2016), where complex multi-stage compression is utilized. Furthermore, it is noteworthy that the `nn.Module` in the PyTorch framework—the industry standard for AI research—is built upon this exact inheritance structure to manage complex neural architectures.

5. Related Work

Han et al. (2016) proposed the "Deep Compression" pipeline, which sequentially applies pruning, trained quantization, and Huffman coding. Implementing such a complex, multi-stage pipeline using a procedural approach would result in an unmaintainable system due to the exponential increase in conditional logic. By utilizing the inheritance structure demonstrated in this report, researchers can encapsulate each technique into independent modules. This allows for the flexible assembly of compression pipelines, akin to building with Lego blocks,

without compromising the integrity of the original model.

6. Reference

Han, S., Mao, H., & Dally, W. J. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In International Conference on Learning Representations (ICLR).